A Measured Matheuristic Layer for Integrated Air Cargo Schedule Optimization at Integrator Scale

Carlos Soto*

September 2026 — preprint

Abstract

The integrated air cargo scheduling problem (ACSP) of Derigs and Friederichs (OR Spectrum 35:325–362, 2013) jointly selects flights, assigns fleets, builds aircraft rotations and routes cargo over a cyclic week, and is solved exactly by branch-and-price-and-cut at the scale of a few thousand shipments. We study what happens one order of magnitude higher — a synthetic integrator-scale instance with 29,819 origin–destination shipments, 1,250 flights and 154 aircraft in six fleets — where we observe that the integer layer of the exact method stalls completely: across repeated 90-minute runs, the best whole-network search never improves its incumbent. We describe a matheuristic layer that restores progress while keeping the exact machinery where it works: a greedy cover seed monetized by a single warm LP over the generated columns; incumbent improvement through an escalating local-branching ball; a *deliver-all* service model that prices contracted external delivery as recourse columns; and a geographic fix-and-optimize decomposition whose cross-boundary shipments carry *exact* connection windows read from the frozen timetable, so every regional improvement splices back into a globally verified schedule and the cycle is monotone by construction. Every component is toggleable and measured by ablation, including three negative results we report with equal prominence. On the integrator-scale instance the regional cycle gains 4–5 M of weekly profit (synthetic currency units) in minutes of block time where the whole-network search gains nothing in ninety; parallelizing the pricing sweep reduces it from ∼25 s to ∼1 s per iteration and exposes the master LP re-solve as the next bottleneck. All results are single-seed and exploratory; we state this and the road to a full experimental battery explicitly.

1 Introduction

Air cargo networks are planned in layers — flight selection, fleet assignment, rotation planning, cargo routing — that interact strongly: a flight is worth flying only if cargo can reach it, cargo can reach it only if rotations position an aircraft, and an aircraft is available only if the rest of the schedule releases it. Derigs and Friederichs [1] formulated the integrated problem (ACSP) over a cyclic week, gave a path/string column formulation, and solved it by branch-and-price-and-cut (B&P&C) with problem-specific branching and implied-bound cuts, demonstrating the value of integration on four synthetic airline archetypes with up to a few thousand shipments.

This paper asks an engineering question with methodological consequences: *what does it take to keep that integrated optimization useful one order of magnitude higher*, at the scale of a global express integrator — tens of thousands of shipments per week, a thousand-plus flights, a heterogeneous fleet — and what parts of the exact machinery still earn their keep there?

*Independent researcher. Contact: `c.sotogd@gmail.com`. Code, instance generators and all experiment logs accompany this preprint; the repository URL will be added upon publication and is available from the author meanwhile.

Our answers are empirical. We reimplemented the full method of [1] (Section 2), generated instances from reimplementations of the paper's four archetype recipes plus a new public integrator-scale family, and then measured our way through a sequence of additions, keeping what worked and reporting what did not. The result is a matheuristic in the modern sense [13, 14]: heuristic scaffolding — greedy construction, large-neighborhood integer moves, spatial fix-and-optimize — around an exact column-generation engine that supplies the three things heuristics cannot: feasible *columns* for a timed, capacitated network; *dual prices* that tell every heuristic where value lies; and *valid bounds* that certify how much is left on the table.

Contributions.

1. A *deliver-all* service model for integrators (Section 3): demand rows become equalities and every shipment carries an always-available contracted-delivery column priced at a multiple of own flying economics, derived from the mandatory schedule only so the tariff is identical across all model variants. Every relaxation, sub-MIP and decomposition block becomes feasible by construction, and the optimization becomes "own network versus contracting bill".

2. An integer-adoption pipeline for huge masters (Section 4): greedy cover seeding, flow monetization by a single warm LP over the freshly generated column pool, and an escalating local-branching ball [5] as the primal heuristic, with a per-round observable (*LP promise vs. integer adoption*) that separates "the candidate generator is weak" from "integer adoption is the bottleneck".

3. A geographic fix-and-optimize decomposition (Section 6) whose distinguishing features are *exact* gateway windows read from the frozen timetable (no estimated connection times), a balance-repair partition that keeps both the frozen and the re-optimized side assemblable into rotations, an exact fleet slice with a learned correction for chain-splitting rounding, and a monotone merge guard: a block result is accepted only when the independently verified *global* schedule improves. An optional pairwise "relay" re-opens cross-region demand with no cross-pass bookkeeping.

4. A measured engineering layer (Section 7): a bit-identical parallel pricing sweep, Farley-type bounds [9] under truncation with a membership filter, and per-iteration cost breakdowns that make the next bottleneck visible instead of guessed. Mispricing-safe dual smoothing [7, 8] is implemented and correctness-verified; its performance evaluation belongs to the planned battery and no benefit is claimed here.

5. An honest computational study (Section 8) on public, deterministic, regenerable instances, including three negative results: coordinated "hub wave" candidate bundles that the optimizer never adopts; cross-region relay passes that run clean but earn nothing at the chosen recourse tariff; and a specialized LP method (sifting) that loses to warm-started dual simplex on the colgen master.

We are explicit about scope (Section 9): headline numbers are single-seed, the instances are synthetic with AI-estimated parameters, and comparability with [1] is qualitative because neither their instances nor their code were published. This preprint plants the system, the mechanisms and the measurement discipline; the multi-seed battery and an arc-flow baseline are the declared next step.

2 Problem and base algorithm

We summarize [1] only to fix vocabulary. A weekly cyclic horizon of $N = 10{,}080$ minutes carries: *airports* (some transfer hubs with minimum connection times, transfer and storage costs); *fleets*

airports / transfer hubs	150 / 9
fleets (aircraft)	6 (154)
flights in base schedule (mandatory / optional)	1,250 (625 / 625)
shipments (O&D demands)	29,819
total demand	~62.7 kt/week
service commitment	deliver-all (Section 3)

Table 1: The integrator-scale instance RLA-I-s1 (synthetic, seed 1).

with counts, payload/volume capacities and costs; *flights* (sequences of legs; mandatory or optional; possibly external capacity with booking costs); and *shipments* (O&D demands with release times, delivery deadlines, tonnage, volume, freight rate).

Model ACSP-T is a path/string formulation over a time-space network: columns are cargo *paths* (feasible timed itineraries for one O&D) and flight *strings* (fleet-assigned flight sequences; in the FARP-T variant used throughout this paper a string is a single flight and rotations are chained through ground arcs). Rows enforce demand limits, leg payload and volume capacities coupled to string selection, flight cover, per-fleet flow balance at timeline events, and fleet counts. Columns are priced by two oracles: PRICE-P, a resource-constrained shortest path solved with A*; and PRICE-S for strings. Implied-bound cuts link flow to selection; problem-specific branching (flight in/out, fleet-flight, follow-on) keeps both pricers intact. Without maintenance the procedure is exact. Our reimplementation follows the paper; every accepted incumbent is re-validated by an independent feasibility checker that shares no code with the solver.

Instances. We reimplemented the generation recipes of the four archetypes of [1] (regional, international, mixed, express) from the published parameter tables, and added two fictional families: GI ("global integrator": 4 hubs, 140 aircraft, dense demand) and RLA ("real-life-scale airline": 9 hubs on three continents, 154 aircraft in six fleets, gravity-model demand on 60% of ordered station pairs with weekly lane recurrence, 1,250-flight base schedule of which half is mandatory, and 29,819 shipments). All parameters are AI-estimated for plausibility; no real airline data is used. Generators are deterministic in an integer seed and public. Table 1 summarizes the instance RLA-I-s1 used for the scale experiments.

3 A deliver-all service model

Integrators do not cherry-pick profitable freight; a service commitment means everything tendered is delivered, if necessary by a contracted external carrier. We model this by (i) turning demand rows into equalities and (ii) giving every shipment od a *contracted delivery column* with objective coefficient $r_{od} - c^{\text{rec}}_{od}$, upper bound d_{od}, and a single nonzero entry in the demand row. The recourse tariff is

$$c^{\text{rec}}_{od} \;=\; \mu \cdot \big(\delta_{od}\, v + \delta_{od}\, \phi / W\big),$$

with δ_{od} the great-circle distance, v the average own variable cost per tonne-km, ϕ the median own fixed cost per km, W the largest fleet payload, and $\mu = 3$ throughout. Crucially, v and ϕ are derived from the *mandatory* flights only: that set is identical in every model variant of a design or decomposition run (optional candidates come and go), so the tariff — and hence the profit scale — is stable across all comparisons. An earlier version derived the tariff from all flights and suffered a ~1% per-round drift that silently invalidated round-over-round comparisons.

Three consequences matter. Revenue becomes a constant, so the problem is a cost minimization (own flying vs. contracting bill). Every selection of flights is completable, so seeds, restricted sub-MIPs and decomposition blocks are feasible by construction — the entire pipeline of Sections 4 and 6 leans on this. And the recourse coefficient caps the demand duals at the tariff, a free mild stabilization of the master.

4 Integer adoption at scale

At integrator scale the practical failure is primal: branch-and-bound explores 1–3 nodes within realistic budgets (Section 8.1), so incumbents must come from elsewhere. Our pipeline has four stages, each measured in isolation in Table 2.

(a) Greedy cover seed. A constructive pass assigns every mandatory flight to its lightest compatible fleet as balanced round trips (inter-hub mirror pairs kept together), followed by an evaluated repair loop. The seed is feasible but flowless: it earns no revenue (deliver-all: everything is contracted).

(b) Seed flow loader. After the root column generation, the seed's selection is fixed in the master and *one* warm LP optimizes cargo flows over the column pool the pricers just generated. This is where column generation pays its first dividend: the pool contains thousands of priced itineraries, and one LP monetizes the skeleton in seconds. No new pricing is needed.

(c) Escalating local-branching ball. The primal heuristic solves the restricted master MIP inside a Hamming ball of at most k selection flips around the incumbent [5], escalating $k \to 2k \to 4k$ (default $k = 60$) while flat within a budget slice. The incumbent is always feasible inside the ball, so the step is monotone; the solver, not a hand-built neighborhood, chooses *which* flips, which keeps cross-network rotation rewirings reachable. An unrestricted restricted-master MIP at this scale finds nothing in equal budget (Table 2); the ball turns the same budget into adopted profit. On adoption, flows are re-monetized by one further warm LP.

(d) Promise vs. adoption. Each design round logs the LP bound uplift contributed by the current candidate batch (*promise*) against the integer profit actually adopted. This single observable separated two hypotheses we could not otherwise distinguish — "the proposer generates worthless candidates" vs. "the integer layer cannot digest them" — and later diagnosed a genuine economic no (Section 8.4).

5 The autonomous design loop

The base algorithm assumes a human writes the optional-flight candidate list. Our design loop generates it: each round proposes a batch of candidate flights aimed at unserved demand (hub round trips, direct rotations, inter-hub trunks, premium external capacity), solves the extended instance with warm-started columns and a seeded incumbent, scores which candidates flew, and evicts persistent non-flyers (with amnesty). Two scale devices are toggleable: consolidation of tiny, far shipments into hub-corridor pseudo demands during the rounds (the final solve always runs on the original demand), and zone rotation of proposal targeting. We omit details for space; all of it is in the repository's technical report. One addition is a negative result we keep in the codebase, disabled: *hub waves*, coordinated feeder–trunk–distribution bundles proposed as a set for the long tail of scattered small shipments. The optimizer never adopted one (Section 8.4).

6 Geographic fix-and-optimize with exact gateway windows

Motivated by the observation that block-sized models adopt improvements in seconds while the whole-network master adopts nothing (Section 8.3), we decompose the planet: hub clusters (hubs within 3,000 km) define regions; every airport joins its nearest hub's region.

Blocks. A block re-optimizes one region against a frozen world. Re-decidable (*kept*) flights are those with every leg inside the region, minus three repair classes computed to a fixpoint: (i) *balance repair* — the kept *selected* strings must balance arrivals and departures per (fleet, airport), or neither side of the partition assembles into rotations; unbalanced one-way strings (typically a multi-stop trunk whose mirror has an en-route stop in a third region) are frozen one at a time, so the cascade follows the unbalanced chain and stops; (ii) flows whose path crosses the region more than once, and (iii) degenerate cuts (a loop entering and leaving at the same airport), whose flights are frozen because the merge splices exactly one regional segment per donated flow.

Exact windows. Cross-boundary cargo is truncated at its gateway with windows read from the *frozen timetable*, not estimated: an outbound segment must end before its frozen onward connection departs minus the gateway transfer time; an inbound segment becomes available when its frozen feeder lands plus transfer. Cargo handling at true origins and destinations is encoded into the windows so gateway hand-offs do not pay it twice. Consequently a time-feasible block solution *splices* back into a time-feasible global schedule by construction. Shipments with both endpoints inside the region enter whole; under deliver-all their contracted remainder rides along, so every block is feasible from its seed.

Fleet slice. The block's fleet budget is count−used(all)+used(kept), computed by assembling the kept side alone (possible after balance repair). Splitting mixed rotation chains can cost a rounding aircraft per fragment that no a-priori budget foresees; instead of estimating it, the cycle *learns* it: a merged block that fails the global fleet-size check by e_k aircraft teaches its block a budget reduction of e_k for the next visit.

Monotone merge. A block solution is spliced into the global schedule (frozen strings and flows kept; donated segments replaced; sub-contracted remainders fall back to global end-to-end contracting) and accepted *only if* the independently verified global profit improves. The cycle can therefore never lose ground. During development this guard caught four distinct merge bugs — double-counted multi-crossing segments, duplicated whole-region demands, unassemblable partitions, fleet-rounding violations — without ever letting a corrupted schedule through; we consider it the load-bearing design decision of the whole section.

Pairwise relay. Optionally, after the single-region passes of each cycle, *pair* passes run on region unions ordered by open cross-tonnage. A cross-region shipment enters a pair block whole, with exact windows; feeder, trunk and distribution are decided in one model, avoiding any cross-pass ledger or estimated hand-off times.

7 Engineering for honest measurement

Parallel pricing, bit-identical. Each pricing iteration asks one independent shortest-path question per shipment against the same duals. The sweep runs one shipment per core and collects results in shipment order, so its output is bit-identical to the sequential sweep (asserted by test and verified in production runs: identical column counts and bounds). The per-destination A* bounds — the only lazily built shared state — are prewarmed one Dijkstra pair per core. The servability probes of the design loop parallelize the same way with thread-local dual vectors.

Mispricing-safe dual smoothing. Optionally, pricers see $\alpha \cdot \text{center} + (1-\alpha) \cdot \text{duals}$ [7, 8] ($\alpha = 0.7$). Two safeguards keep exactness: a smoothed pass that finds no columns triggers a re-price with the true duals before any conclusion, and validity-critical bounds are computed from true-dual passes only. A test asserts that the stabilized root converges to the same LP value as the plain one. We report no performance numbers for smoothing: it is implemented, verified and off by default, and its evaluation is deliberately left to the multi-seed battery — we flag this to avoid claiming a benefit we have not measured.

Bounds under truncation. Hard per-round budgets are made mathematically legal by a Farley-type bound [9]: on interruption the tightest valid bound seen across full pricing passes is returned, each computed as the RMP value plus the most optimistic contribution of *missing* columns. A membership filter is essential in practice: pricers legitimately return columns already in the master (nonbasic at bounds, positive reported reduced cost); counting them produces absurd bounds. Only columns not yet in the master contribute.

Cost visibility. Every colgen iteration reports its breakdown (pricing / column loading / LP re-solve seconds). This 20-line addition settled in minutes a question we had been guessing at for days (Section 8.5).

8 Computational results

Protocol and disclosure. All numbers below are from single runs on one machine (Apple Silicon, 10 cores; CPLEX 22.2 as LP/MIP backend, HiGHS as fallback), on instance RLA-I-s1 unless stated. They are exploratory: no variance estimates, one seed. We report them because their margins are large and their patterns repeated across the campaign; the multi-seed battery over all instance families is the declared next step. All runs are reproducible from the public repository (deterministic seeds; every figure's raw log is committed).

8.1 The whole-network integer layer stalls

From a shared incumbent at 139.7–139.8 M (produced by cover seed + loader), the whole-network solver — warm-started column pool, seeded incumbent, escalating ball, 90-minute budget — produced *zero* incumbent improvements, twice (independent runs at 1,800 s and 5,400 s produced the same +0). Branch-and-bound explored 1–3 nodes. This is the phenomenon the rest of the paper responds to: at this scale the exact tree is vestigial and even ball-restricted sub-MIPs drown in the full master's LP.

8.2 The adoption pipeline

Table 2 shows the round-0 waterfall on the consolidated design model (25,451 demands after tiny-far consolidation). Each stage is enabled on top of the previous.

The best full-demand schedule obtained by the design pipeline alone reaches 142.8 M at a 9.9% certified gap; in it, 22,011 of 29,819 shipments (46.0 kt, a 46.6 M bill) are contracted and only 58 of 154 aircraft fly. That anatomy — a long tail of ∼2-tonne shipments scattered over ∼150 stations — motivates both the wave experiment (negative, below) and the decomposition (positive, next).

stage	profit (M)	increment
greedy cover seed (all demand contracted)	129.9	—
+ seed flow loader (one warm LP over the pool)	139.6	+9.7
+ local-branching ball k=60	142.3	+2.7
+ adoption re-monetization (one warm LP)	143.3 / 143.5	+1.0
unrestricted restricted-master MIP, same budget	139.6	+0.0

Table 2: Round-0 incumbent waterfall on RLA-I-s1 (consolidated model, deliver-all). The re-monetization column shows two independent runs. The last row is the ablation: without the ball, the same heuristic budget adopts nothing.

arm	budget	gain	note
A: whole-network continuation	1,800 s	+0	
A: whole-network continuation	5,400 s	+0	independent repeat
B: regional cycle	900 s	+5.03 M	6 accepted blocks, then flat
B: regional cycle (learned fleet shave)	1,800 s	+3.96 M	includes learning visits

Table 3: Whole-network vs. regional split from a shared incumbent (RLA-I-s1, full 29,819-shipment demand). Blocks of 200–350 flights adopt improvements in 4–50 seconds each.

8.3 Geographic decomposition

A/B protocol: one shared base incumbent, then the same additional budget spent either (A) continuing the whole-network solve or (B) running the regional cycle. Table 3 summarizes four consecutive runs (two clean A/Bs plus two earlier runs whose arm B was invalidated by merge bugs — in *every* run, including the buggy ones, the global feasibility guard held: no corrupted schedule was ever accepted).

The mechanism, not just the outcome, matches the diagnosis: all four regions adopt on their first visit (from +0.26 M to +1.5 M each, in 9–50 s of block time), improvements shrink on the second cycle, and the cycle then honestly reports flat blocks until the budget closes. The best full-demand schedule of the campaign (144.8 M) came from a 5-minute base solve plus a 15-minute regional cycle.

8.4 Negative results

Hub waves. A fifth candidate class proposed coordinated feeder–trunk–distribution bundles per (hub corridor, day), with chain-consistent times and honestly windowed tonnage. Over 20 design rounds (6,000 candidates, waves included at 30% quota), *zero* were adopted, and the LP promise of candidate batches decayed from +4.4 M (round 1) to ∼0 (round 3 on). *Pair relay.* The pairwise passes of Section 6 ran clean (no crashes, all merges verified) and adopted nothing at $\mu = 3$. *Interpretation.* Three independent probes — waves, promise decay, pairs — agree: at triple own economics, the scattered cross-region tail is genuinely cheaper to contract than to fly. This is a statement about the instance's economics, not about the algorithms; both mechanisms remain in the code, disabled by default, for cheaper-recourse or larger-scale settings.

LP method. On the colgen master (> 40k columns), CPLEX sifting — nominally suited to column-heavy LPs — measured 2–3× *slower* per re-solve (49–114 s) than warm-started dual simplex (18–44 s). The advantage of basis reuse across colgen iterations is well known; the value here is quantifying it for this master, since it rules out a drop-in remedy for the LP bottleneck of Section 8.5.

8.5 Parallel pricing and the next bottleneck

Parallelizing the pricing sweep cut it from $\sim$25 s to 0.4–1.0 s per iteration (identical columns and bounds), but end-to-end root time improved only $\sim$15% (156 s $\rightarrow$ 133 s for six iterations). The per-iteration breakdown explains it: pricing 1.0 s, column loading 0.1 s, master LP re-solve 18–44 s and growing with the column count. Amdahl's ledger is now unambiguous: the master LP is the next bottleneck, and the natural lever is column deletion (keeping the master lean) rather than a faster pricer. Dual smoothing also gains value from this shift: every avoided iteration now saves a full LP re-solve.

9 Limitations

(i) Headline results are single-seed, single-machine, one instance family at scale; margins are large but variances are unmeasured. (ii) All instances are synthetic; parameters (costs, demands, fleets, curfews, payload-range curves) are AI-estimated for plausibility and calibrated in scale — not in detail — to [1]; no real airline data is used or implied. (iii) Comparability with [1] is qualitative only: their instances and code were not published, so our reimplementation of their generators matches parameters, not files. (iv) There is no external baseline yet (e.g., a direct arc-flow MIP under equal budget); "better than our own previous configuration" is the only comparison this preprint supports. (v) The design loop is greedy across rounds; inner solves carry valid bounds, but no global optimality claim spans rounds.

10 Conclusions and road map

At integrator scale, the exact tree of B&P&C contributes almost nothing, but the column-generation engine remains the value chain's spine: every profitable move in our campaign flowed through priced columns, warm LPs over them, or dual signals derived from them. The productive question is not "exact or heuristic" but *where each layer earns its keep*: LP/colgen for columns, prices and certificates; local branching for integer moves the full master cannot digest; geographic fix-and-optimize for turning block-sized tractability into global, verified, monotone progress.

Road map, in order: (1) the multi-seed battery over all instance families with component ablations; (2) an arc-flow MIP baseline under equal budget; (3) column deletion for the master LP; (4) deeper trees inside regional blocks, where nodes cost seconds; (5) any form of real-data validation, which would upgrade every claim in this paper.

Reproducibility and AI disclosure

The complete implementation (solver, generators, web workbench, 120 verification tests), all instance generators, and the raw logs behind every number in this paper are public in the accompanying repository. Runs are deterministic given a seed. This project was developed with substantial assistance from an AI coding agent (Anthropic Claude), including code, instance parameter estimation, experiment execution and the drafting of this manuscript; all algorithmic decisions, measurements and claims were verified by the reproducible pipeline above, and the human author reviewed and takes responsibility for the content. Airline names and data are fictional.

References

[1] Derigs, U., Friederichs, S.: Air cargo scheduling: integrated models and solution procedures. OR Spectrum 35, 325–362 (2013).

[2] Barnhart, C., Johnson, E.L., Nemhauser, G.L., Savelsbergh, M.W.P., Vance, P.H.: Branch-and-price: column generation for solving huge integer programs. Operations Research 46(3), 316–329 (1998).

[3] Lübbecke, M.E., Desrosiers, J.: Selected topics in column generation. Operations Research 53(6), 1007–1023 (2005).

[4] Desaulniers, G., Desrosiers, J., Solomon, M.M. (eds.): Column Generation. Springer (2005).

[5] Fischetti, M., Lodi, A.: Local branching. Mathematical Programming 98, 23–47 (2003).

[6] Danna, E., Rothberg, E., Le Pape, C.: Exploring relaxation induced neighborhoods to improve MIP solutions. Mathematical Programming 102, 71–90 (2005).

[7] Wentges, P.: Weighted Dantzig–Wolfe decomposition for linear mixed-integer programming. International Transactions in Operational Research 4(2), 151–162 (1997).

[8] du Merle, O., Villeneuve, D., Desrosiers, J., Hansen, P.: Stabilized column generation. Discrete Mathematics 194, 229–237 (1999).

[9] Farley, A.A.: A note on bounding a class of linear programming problems, including cutting stock problems. Operations Research 38(5), 922–923 (1990).

[10] Shaw, P.: Using constraint programming and local search methods to solve vehicle routing problems. In: Principles and Practice of Constraint Programming (CP), 417–431 (1998).

[11] Ropke, S., Pisinger, D.: An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transportation Science 40(4), 455–472 (2006).

[12] Helber, S., Sahling, F.: A fix-and-optimize approach for the multi-level capacitated lot sizing problem. International Journal of Production Economics 123, 247–256 (2010).

[13] Archetti, C., Speranza, M.G.: A survey on matheuristics for routing problems. EURO Journal on Computational Optimization 2, 223–246 (2014).

[14] Fischetti, M., Fischetti, M.: Matheuristics. In: Martí, R., Pardalos, P.M., Resende, M.G.C. (eds.) Handbook of Heuristics. Springer (2018).

[15] Crainic, T.G.: Service network design in freight transportation. European Journal of Operational Research 122, 272–288 (2000).

[16] Feng, B., Li, Y., Shen, Z.-J.M.: Air cargo operations: literature review and comparison with practice. European Journal of Operational Research 247(3), 673–685 (2015).

[17] Brandt, F., Nickel, S.: The air cargo load planning problem — a consolidated problem definition and literature review. European Journal of Operational Research 275(2), 399–410 (2019).

[18] Feillet, D.: A tutorial on column generation and branch-and-price for vehicle routing problems. 4OR 8, 407–424 (2010).

[19] Huangfu, Q., Hall, J.A.J.: Parallelizing the dual revised simplex method. Mathematical Programming Computation 10, 119–142 (2018).